



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1

System Integration and Architecture

System Design Document (SDD)

Project Title: BoduBodu - Bodyweight Exercise

Prepared By: Hisoler, Rhegynne Leighlane Monsanto

Version: 1.3

Date: 5/27/2026

Status: Final

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	2/21/2026	Rhegynne Hisoler	Initial draft	Draft
1.0	2/28/2026	Rhegynne Hisoler	Finalizing all Contents	Final
1.1	5/13/2026	Rhegynne Hisoler	Website UI	Draft
1.2	5/13/2026	Rhegynne Hisoler	Mobile UI	Draft
1.3	5/25/2026	Rhegynne Hisoler	Updated SDD contents to reflect the implemented backend, web, mobile, API, and database details	Updated Draft
1.4	5/27/2026	Rhegynne Hisoler	Website and Mobile	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

BoduBodu – Bodyweight Exercise is a fitness system composed of a Spring Boot backend API, a React web application, and a native Android mobile application. The current implementation supports email/password authentication, Google sign-in, password reset flow, user profile management, exercise browsing, predefined workouts, custom workout creation, workout logging, workout history, progress summaries, and admin management features.

The backend exposes REST endpoints secured with JWT bearer tokens and role-based access control. The web client stores the token and user profile in localStorage and routes users through protected React Router views. The Android client uses Retrofit and Gson to call the same backend endpoints.

BoduBodu focuses on simplicity, accessibility, and user motivation, making bodyweight training structured, trackable, and beginner-friendly.

1.2 Objectives

1. Develop a fully functional bodyweight exercise platform with JWT authentication, Google sign-in, password reset, and workout management
2. Implement a three-tier architecture using Spring Boot 4, ReactJS, and native Android Kotlin
3. Use RESTful APIs for authentication, user profile, exercises, predefined workouts, custom workouts, dashboard stats, workout logging, and workout history
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Enable users to track completed workouts and monitor fitness progress
6. Enable users to create and customize their own workout routines based on personal fitness goals.
7. Provide admin-only management for users, exercises, and predefined workouts.
8. Keep the system ready for deployment by using environment-based configuration for database, Google OAuth, mail, and reset-password settings

1.3 Scope

Included Features:

- User registration, login, Google authentication, forgot password, reset password, and JWT-secured sessions
- Authenticated user profile retrieval and profile update, including optional password change
- Exercise library containing bodyweight exercises with descriptions, difficulty levels, target muscle group, and optional video field
- Predefined workout listing through the implemented default workout module
- Categorized workout programs (Beginner, Intermediate, Advanced)
- Custom workout creation (users can create and modify their own workout routines)
- Workout session tracking (start workout, mark exercises as completed, finish session)
- Workout history and progress tracking
- Admin panel for managing users, exercises, and predefined workouts
- Responsive web application
- Native Android mobile application
- PostgreSQL relational database configuration through Spring Data JPA

Excluded Features:

- Integration with wearable fitness devices (e.g., smartwatches)
- AI-generated personalized workout plans
- Real-time video coaching or live streaming
- General email notification system beyond password reset
- Social media sharing integrations
- Push notifications
- Paid subscription or payment gateway integration
- Advanced fitness analytics and performance prediction

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive System Design Document (SDD) for the BoduBodu Bodyweight Exercise system as implemented in the current repository. It defines the actual backend, web, mobile, database, API, and UI/UX design details.

The purpose of this document is to provide a clear technical blueprint for maintaining and presenting the system across its Spring Boot API, React web client, Android mobile client, and PostgreSQL database.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: BoduBodu - Bodyweight Exercise

Domain: Health/Fitness

Primary Users:

1. General Users (Fitness Users)
2. Administrators

Problem Statement: Many individuals want to stay active but do not have access to gym equipment or prefer equipment-free training. **Solution:** BoduBodu provides a cross-platform fitness system with structured bodyweight workouts, custom routines, workout logging, progress summaries, and admin management through web and mobile applications.

2.2 Core User Journeys

Journey 1: New User Completing a First Workout

1. A new user accesses the web or mobile application.
2. The user registers using email/password, signs in with Google, or logs in with existing credentials.
3. Available predefined workout programs are loaded from `/api/v1/default-workouts`.
4. A workout program is selected and its exercises are reviewed.
5. The workout session starts.
6. Exercises are completed and the session is finished.

7. The completed workout is logged through /api/dashboard/log-workout and appears in workout history.

Journey 2: Creating a Custom Workout

1. The user logs into the system.
2. The custom workout section is accessed through the web Custom Workouts page or Android custom workout screens.
3. Exercises are selected from the exercise library.
4. Repetitions, sets, and rest intervals are configured.
5. The custom workout is saved through /api/user/custom-workouts.
6. The workout is started and completed.
7. The session is recorded in workout history and dashboard statistics.

Journey 3: Tracking Progress

1. The user navigates to the dashboard, progress page, profile page, or workout history section.
2. Previously completed workouts are displayed.
3. Details such as completion date, type, duration, exercise count, weekly totals, and streak values are reviewed.
4. The user may begin another predefined or custom workout.

Journey 4: Administrator Managing System Data

1. The administrator logs in with authorized credentials.
2. The admin dashboard and user-management screens are accessed.
3. Users, exercises, and predefined workouts are viewed, added, edited, or removed according to admin permissions.
4. Changes are reflected immediately in the system.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, Google sign-in, forgot password, reset password, logout on client)
2. Exercise library (view exercises with details)
3. Predefined workout programs loaded from the default workout module
4. Custom workout creation and editing
5. Workout session tracking (start, complete, record duration)
6. Workout history tracking
7. Admin panel for user, exercise, and predefined workout management

8. PostgreSQL database integration through Spring Data JPA
9. Secure role-based access control (User / Admin)

SHOULD HAVE

1. Exercise filtering/searching by user-facing fields such as name, muscle group, and difficulty level
2. Progress summary including total workouts, weekly workouts, total minutes, total exercises, current streak, and longest streak
3. Form validation with clear error messages
4. Responsive design across different screen sizes
5. Password recovery via email reset link

COULD HAVE

1. Favorite workouts feature
2. Workout streak tracking and recent-workout summaries
3. Simple progress statistics based on saved workout sessions
4. Minor UI enhancements and animations

WON'T HAVE

1. General notification system beyond password reset email
2. Social media sharing integration
3. Wearable device integration
4. AI-generated workout generation
5. Payment or subscription system
6. Push notifications
7. Advanced admin audit logging

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password, Reset Password
- **Fields:** First Name, Last Name, Email, Password, Confirm Password, Google credential, reset token, new password
- **Validation:** Email format, required names, password requirements, unique email, valid reset token
- **API Endpoints:** POST /api/auth/register, POST /api/auth/login, POST /api/auth/google, POST /api/auth/forgot-password, POST /api/auth/reset-password

- **Security:** JWT tokens, PasswordEncoder hashing, Google ID token verification, password reset token expiry

Feature: Exercise Library

- **Screens:** Exercise Listing, Exercise Detail
- **Display:** Exercise name, description, difficulty level, target muscle group
- **API Endpoints:** GET /api/v1/exercises, GET /api/v1/exercises/{id}
- **Admin Functions:** POST /api/admin/exercises, PUT /api/admin/exercises/{id}, DELETE /api/admin/exercises/{id}

Feature: Predefined Workout Programs

- **Screens:** Workout Listing, Workout Detail
- **Functions:** View workout details, Start workout session
- **Display:** Workout name, difficulty level, list of exercises
- **API Endpoints:** GET /api/v1/default-workouts

Feature: Custom Workout Creation

- **Screens:** Create Custom Workout, Edit Custom Workout
- **Field:** Workout name, selected exercises, repetitions, sets, rest interval
- **Functions:** Add exercises, Remove exercises, Update repetitions and sets, Save custom workout
- **API Endpoints:** POST /api/user/custom-workouts, GET /api/user/custom-workouts, PUT /api/user/custom-workouts/{id}, DELETE /api/user/custom-workouts/{id}

Feature: Workout Session Tracking

- **Screens:** Workout Session, Workout Summary
- **Data Collected:** Workout name, user ID, default/custom workout ID, completion date, duration, exercise count
- **Process:** Start workout, Complete workout, Save session record
- **API Endpoints:** POST /api/dashboard/log-workout, GET /api/dashboard/recent-workouts, GET /api/workout-history

Feature: Dashboard, Progress, and Workout History

- **Screens:** Dashboard, Progress, Workout History, Profile statistics section
- **Display:** Total workouts, workouts this week, total minutes, total exercises, current streak, longest streak, recent workouts, completed sessions

- **API Endpoints:** GET /api/dashboard/stats, GET /api/dashboard/recent-workouts, GET /api/workout-history
- **Data Source:** workout_sessions table filtered by authenticated user

Feature: Admin Panel

- **Screens:** Admin Dashboard, User Management, Exercise Management, Predefined Workout Management
- **Functions:** Add/edit/delete users, exercises, and predefined workouts
- **Access Control:** Admin role required
- **API Endpoints:** /api/admin/users, /api/admin/exercises, /api/admin/default-workouts with ROLE_ADMIN validation

2.5 Acceptance Criteria

AC-1: Successful User Registration

- **Given** a new user is on the registration page
- **When** valid email and password are entered
- **And** the email is not already registered
- **And** the registration form is submitted
- **Then** a new account shall be created
- **And** the user shall be able to log into the system

AC-2: Successful User Login

- **Given** a registered user is on the login page
- **When** correct email and password are entered
- **And** the login form is submitted
- **Then** the system shall authenticate the user
- **And** generate a valid JWT token returned as a token field
- **And** redirect the user to the dashboard or homepage

AC-3: Viewing Exercise Library

- **Given** a logged-in user accesses the exercise section
- **When** the exercise list is loaded
- **Then** all available exercises shall be displayed
- **And** each exercise shall show its name and difficulty level

AC-4: Creating a Custom Workout

- **Given** a logged-in user accesses the custom workout section

- **When** exercises are selected
- **And** repetitions and sets are configured
- **And** the workout is saved
- **Then** the custom workout shall be stored in the database
- **And** be available for future use

AC-5: Completing a Workout Session

- **Given** a logged-in user starts a workout
- **When** the workout is completed
- **And** the session is marked as finished
- **Then** the workout session shall be recorded through /api/dashboard/log-workout
- **And** the completion date and duration shall be saved
- **And** the session shall appear in workout history

AC-6: Viewing Profile and Workout History

- **Given** a logged-in user accesses the profile page
- **When** the page is loaded
- **Then** the system shall display basic account information
- **And** show dashboard statistics such as total workouts, total minutes, exercise count, and streaks
- **And** list past workout sessions

AC-7: Admin Managing Users, Exercises, and Workouts

- **Given** an administrator is logged into the system
- **When** a user, exercise, or predefined workout is added or updated with valid details
- **Then** the record shall be stored in the database
- **And** become visible in the corresponding admin or user-facing screen

- **Given** an administrator edits or deletes a managed record
- **Then** the changes shall be reflected immediately in the system

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time shall not exceed **5 seconds** for 95% of requests under normal load.

- Web pages shall load within **5 seconds** on a stable broadband connection.
- Mobile application cold start time shall not exceed **5 seconds**.
- The system shall support at least **50–100 concurrent users**.
- Database queries shall complete within **500 milliseconds** under normal conditions.
- The system shall support efficient storage and retrieval of exercise data, workout definitions, and completed workout-session records.

3.2 Security Requirements

- Production communications between client and server shall use HTTPS.
- User authentication shall be implemented using **JWT tokens**.
- Passwords shall be hashed using the configured Spring Security PasswordEncoder before storage in the database.
- Role-based access control shall restrict /api/admin/** operations to ROLE_ADMIN.
- The system shall validate user inputs through DTO validation, client-side form checks, and server-side request handling.
- The system shall validate all user inputs before processing.

3.3 Compatibility Requirements

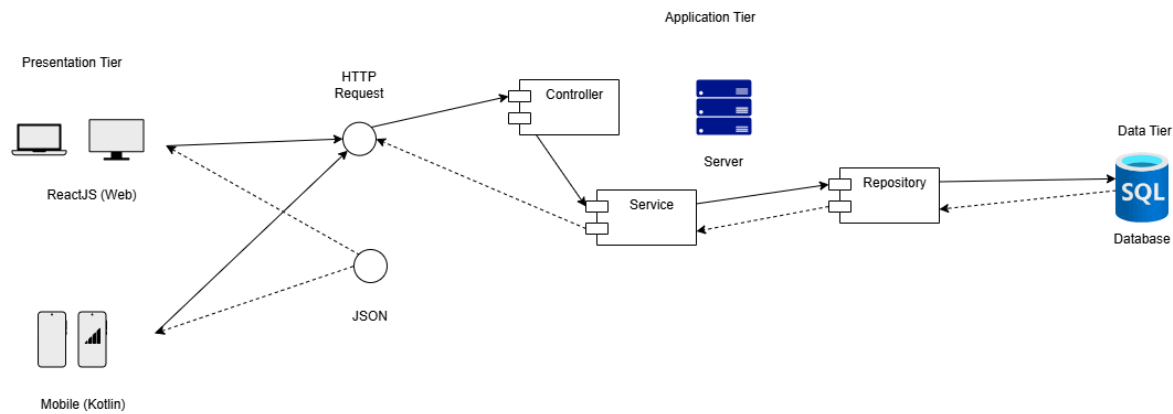
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions) for the React web client
- **Android:** API Level 24+ (minSdk 24, targetSdk 32)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+ for development and browser access

3.4 Usability Requirements

- A new user shall be able to complete their first workout within **5 minutes** of registration.
- Navigation shall remain consistent across all screens.
- Error messages shall clearly describe the issue and provide corrective guidance.
- Buttons and interactive elements on mobile shall meet minimum touch size standards (44x44px).
- Forms shall provide immediate validation feedback.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 4.0.3, Spring Security, Spring Data JPA
- **Database:** PostgreSQL using the configured PostgreSQL driver and datasource
- **Web Frontend:** ReactJS 19, react-router-dom 7, CSS
- **Mobile:** Kotlin Android, Retrofit, Gson converter, Google Play Services Auth
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Backend and web deployable to cloud platforms; Android distributed as APK

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	http://localhost:8080 during local development; endpoint groups are split across /api/auth, /api/user, /api/dashboard, /api/v1, /api/workout-history, and /api/admin.
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Current Response Style	The implemented backend returns direct strings, maps, entity/list objects, and HTTP status codes rather than a universal response envelope.

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Creates a new user account in the system
API URL	/api/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "firstName": "<firstName>", "lastName": "<lastName>", "email": "<email>", "password": "<password>" }</pre>
Response Structure	Success: "User registered successfully." Conflict: "Email is already registered." Bad Request: "Registration failed: <message>"

User Login

Description	User login in the system
API URL	/api/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "<email>", "password": "<password>" }</pre>
Response Structure	<pre>{ "token": "<jwtAccessToken>" }</pre> Unauthorized: "Incorrect email or password."

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

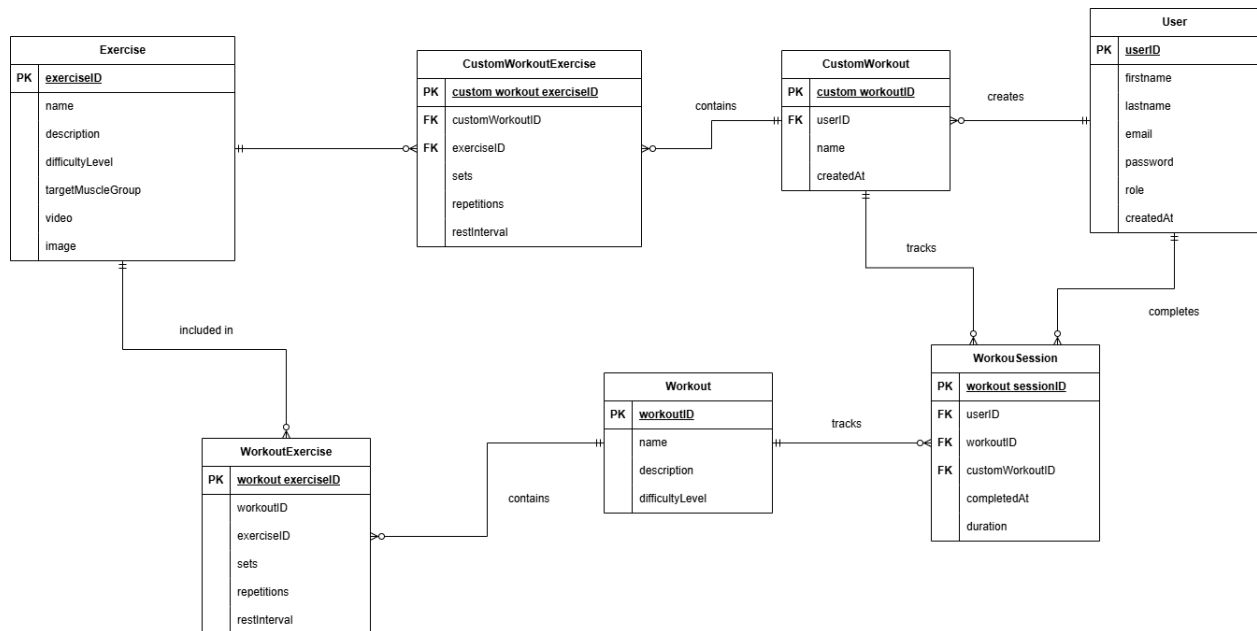
Example 1	{ "token": "<jwt>" }
Example 2	{ "message": "If that email is registered, a password reset link has been sent." }

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- VALID-002: Missing required field
- VALID-003: Invalid input format
- DB-001: Resource not found
- DB-002: Duplicate entry
- DB-003: Database operation failed
- SYSTEM-001: Internal server error
- SYSTEM-002: Unexpected system failure
- WORKOUT-001: Workout not found
- WORKOUT-002: Workout already completed
- SESSION-001: Invalid workout session state
- SESSION-002: Session not found

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many:** User → CustomWorkouts
(Each user can create multiple custom workouts)
- **One-to-Many:** User → WorkoutSessions
(Each user can have multiple completed workout sessions)
- **One-to-Many:** Workout → WorkoutExercises
(Each predefined workout contains multiple exercises)
- **One-to-Many:** CustomWorkout → CustomWorkoutExercises
(Each custom workout contains multiple exercises)
- **Many-to-Many:** Exercise ↔ MuscleGroup
(Resolved through the exercise_muscle_groups junction table)
- **Many-to-One:** WorkoutExercises → Exercise
(Workout exercises reference exercises)
- **Many-to-One:** CustomWorkoutExercises → Exercise
(Custom workout exercises reference exercises)

- **Many-to-One:** WorkoutSessions → User
(Each workout session belongs to one user)
- **Many-to-One:** WorkoutSessions → Workout
(A session may reference a predefined workout)
- **Many-to-One:** WorkoutSessions → CustomWorkout
(A session may reference a custom workout)

Key Tables:

1. **users** - User accounts, roles, password reset fields, and authentication data
2. **exercise** - Bodyweight exercise information
3. **default_workouts** - Predefined workout programs
4. **default_workout_exercises** - Exercises included in predefined workouts
5. **custom_workouts** - User-created workout routines
6. **custom_workout_exercises** - Exercises included in custom workouts
7. **workout_sessions** - Completed workout records used by dashboard, progress, profile, and history screens
- 8.

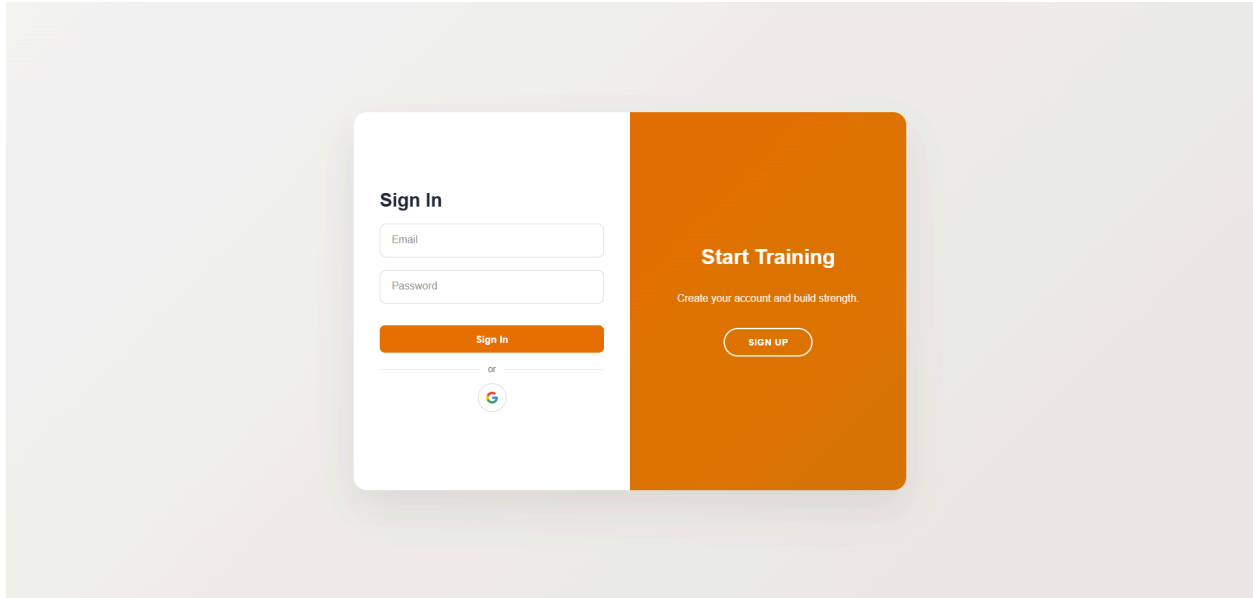
Table Structure Summary:

- **users:** id, first_name, last_name, email, password, role, createdAt
- **exercise:** exerciseId, name, description, difficultyLevel, targetMuscleGroup, video
- **default_workouts:** defaultWorkoutId, name, description, difficultyLevel, createdAt
- **default_workout_exercises:** defaultWorkoutExerciseId, defaultWorkoutId, exerciseId, sets, repetitions, restInterval
- **custom_workouts:** customWorkoutId, userId, name, createdAt
- **custom_workout_exercises:** customWorkoutExerciseId, customWorkoutId, exerciseId, sets, repetitions, restInterval
- **workout_sessions:** id, user_id, workoutName, defaultWorkoutId, customWorkoutId, duration, exercises, completedAt

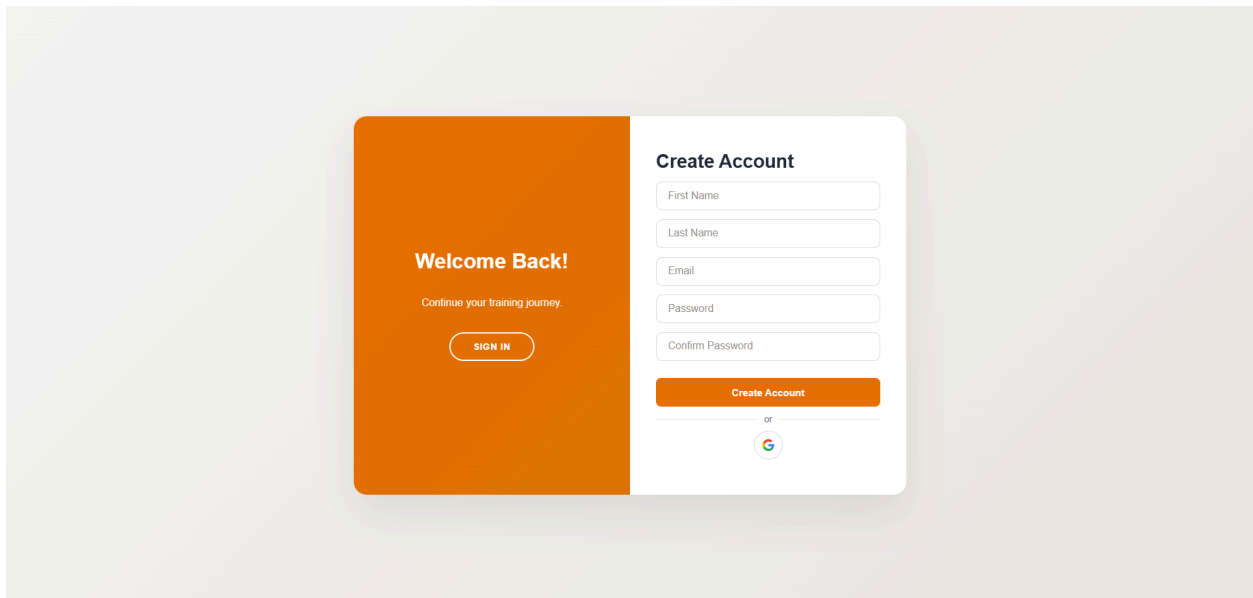
7.0 UI/UX DESIGN

7.1 Web Application Wireframes / Screenshots

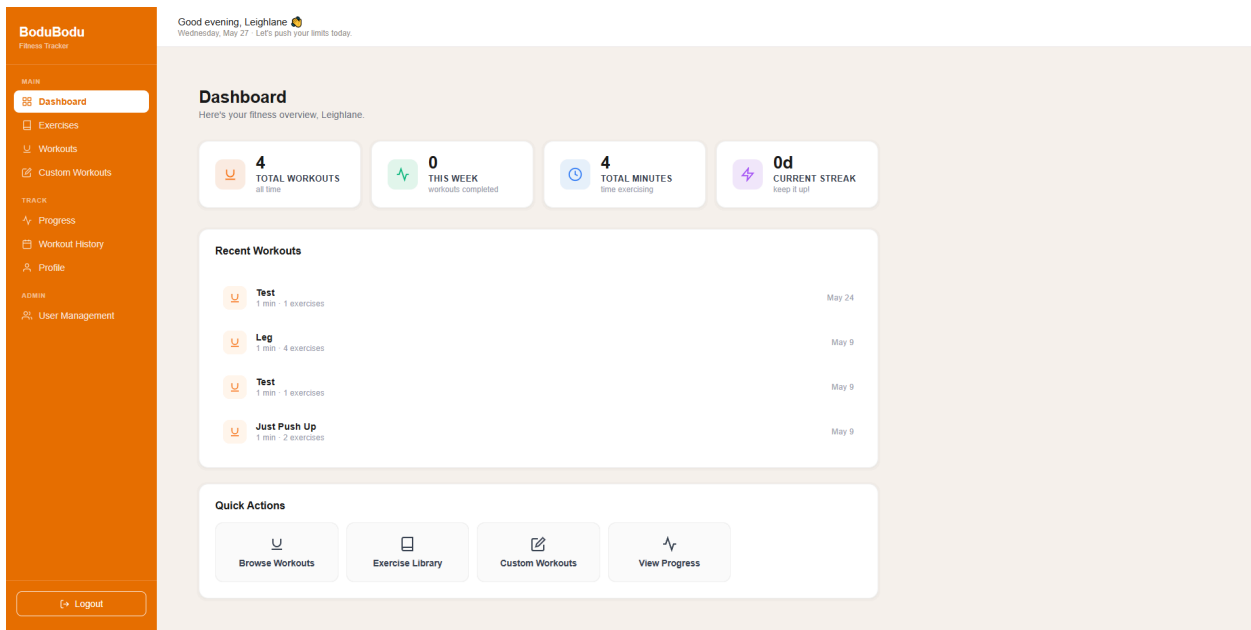
Login Page



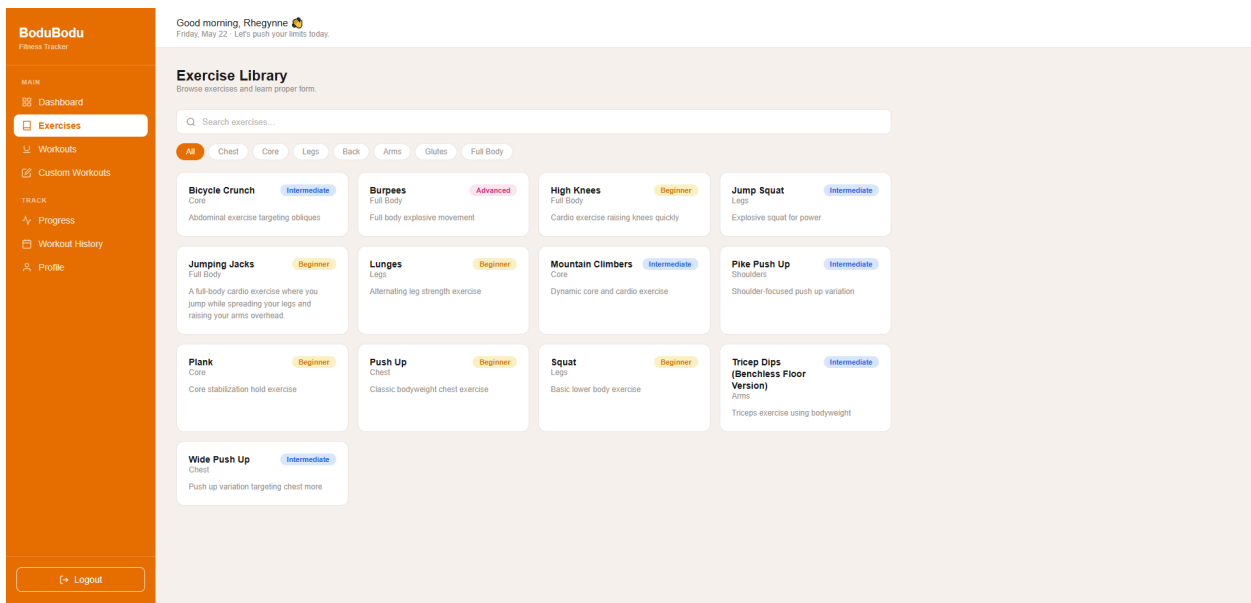
Register Page



Dashboard Page



Exercise Library (User view)



Exercise Library (Admin view)

BoduBodu

Fitness Tracker

MAIN

Dashboard

Exercises

Workouts

Custom Workouts

TRACK

Progress

Workout History

Profile

ADMIN

User Management

Logout

Good morning, Leighlane

Friday, May 22 - Let's push your limits today

ADMIN

Exercise Library

Manage exercises — add, edit, or remove from the library

Search exercises...

AllChestCoreLegsBackArmsGlutesFull Body

Bicycle Crunch

Intermediate

Core

Abdominal exercise targeting obliques

EditDelete

Burpees

Advanced

Full Body

Full body explosive movement

EditDelete

High Knees

Beginner

Full Body

Cardio exercise raising knees quickly

EditDelete

Jump Squat

Intermediate

Legs

Explosive squat for power

EditDelete

Jumping Jacks

Beginner

Full Body

A full-body cardio exercise where you jump while spreading your legs and raising your arms overhead.

EditDelete

Lunges

Beginner

Legs

Alternating leg strength exercise

EditDelete

Mountain Climbers

Intermediate

Core

Dynamic core and cardio exercise

EditDelete

Pike Push Up

Intermediate

Shoulders

Shoulder-focused push up variation

EditDelete

Plank

Beginner

Core

Core stabilization hold exercise

EditDelete

Push Up

Beginner

Chest

Classic bodyweight chest exercise

EditDelete

Squat

Beginner

Legs

Basic lower body exercise

EditDelete

Tricep Dips (Benchless Floor Version)

Intermediate

Arms

Triceps exercise using bodyweight

EditDelete

Wide Push Up

Intermediate

Chest

Push up variation targeting chest more

EditDelete

Logout

Predefined Workouts (User View)

BoduBodu

Fitness Tracker

MAIN

Dashboard

Exercises

Workouts

Custom Workouts

TRACK

Progress

Workout History

Profile

Logout

Good evening, Rhegynne

Wednesday, May 27 - Let's push your limits today

Predefined Workouts

Browse and start predefined workouts

Search workouts...

AllBeginnerIntermediateAdvanced

Core

Intermediate

3 exercises

Build a stronger core and improve stability.

Mountain ClimbersBicycle CrunchBurpees

+2 more

Start

Full Body

Beginner

5 exercises

Easy full-body workout for beginners.

Jumping JacksSquatPush Up

+2 more

Start

Intensity

Intermediate

5 exercises

High intensity workout

Mountain ClimbersBurpeesHigh Knees

+2 more

Start

Leg

Beginner

4 exercises

Strengthen and tone your lower body.

SquatLungesJump Squat

+1 more

Start

Logout

20

Predefined Workouts (Admin View)

BoduBodu

Fitness Tracker

MAIN

Dashboard

Exercises

Workouts

Custom Workouts

TRACK

Progress

Workout History

Profile

ADMIN

User Management

Logout

Good evening, Leighlane

Wednesday, May 27 - Let's push your limits today.

ADMIN

Add Workout

Predefined Workouts

Manage the default workout library — create, edit, or remove workouts.

Search workouts...

All

Beginner

Intermediate

Advanced

Core

3 exercises

Intermediate

Build a stronger core and improve stability.

Mountain ClimbersBicycle Crunch

Burpees

+2 more

StartEditDelete

Full Body

5 exercises

Beginner

Easy full-body workout for beginners.

Jumping JacksSquatPush Up

+2 more

StartEditDelete

Intensity

5 exercises

Intermediate

High intensity workout

Mountain ClimbersBurpees

High Knees

+2 more

StartEditDelete

Leg

4 exercises

Beginner

Strengthen and tone your lower body.

SquatLungesJump Squat

+1 more

StartEditDelete

Custom Workout page

BoduBodu

Fitness Tracker

MAIN

Dashboard

Exercises

Workouts

Custom Workouts

TRACK

Progress

Workout History

Profile

ADMIN

User Management

Logout

Good evening, Leighlane

Wednesday, May 27 - Let's push your limits today.

Custom Workouts

Build and manage your personal workout routines.

Search workouts...

Create new

Date Made

Alphabetical

Test

1 exercise

May 16, 2026

Squat

StartEditDelete

Just Push Up

2 exercises

May 16, 2026

Push UpWide Push Up

StartEditDelete

Experiment

3 exercises

May 9, 2026

Push UpMountain ClimbersPlank

StartEditDelete

Custom Workout Modal

 **Build your Routine** ×

e.g. Morning Strength

BROWSE EXERCISES

Q Search...

☐ Wide Push Up
Intermediate Chest

☐ Plank
Beginner Core

☐ Bicycle Crunch
Intermediate Core

☐ Push Up
Beginner Chest

☐ Squat
Beginner Legs

SELECTED EXERCISES

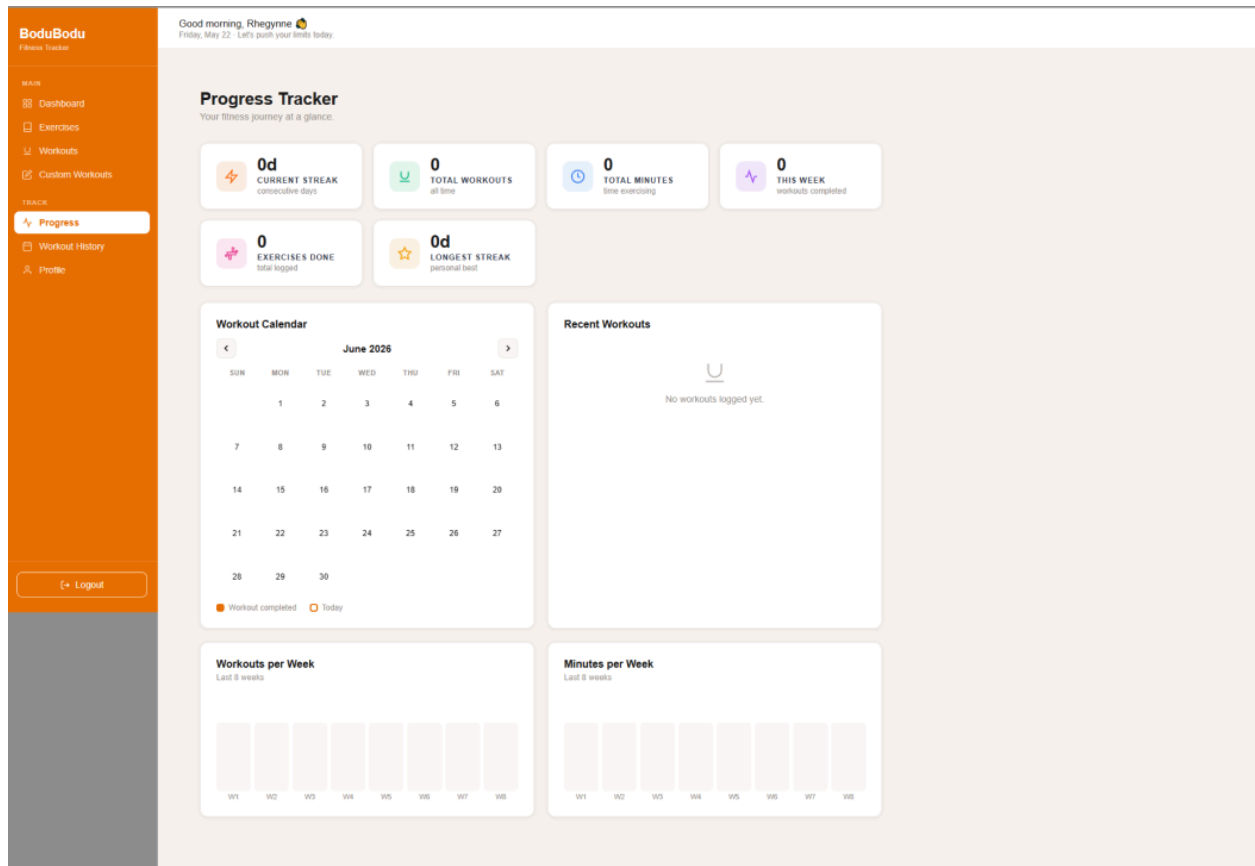
+

Click exercises on the left to add them

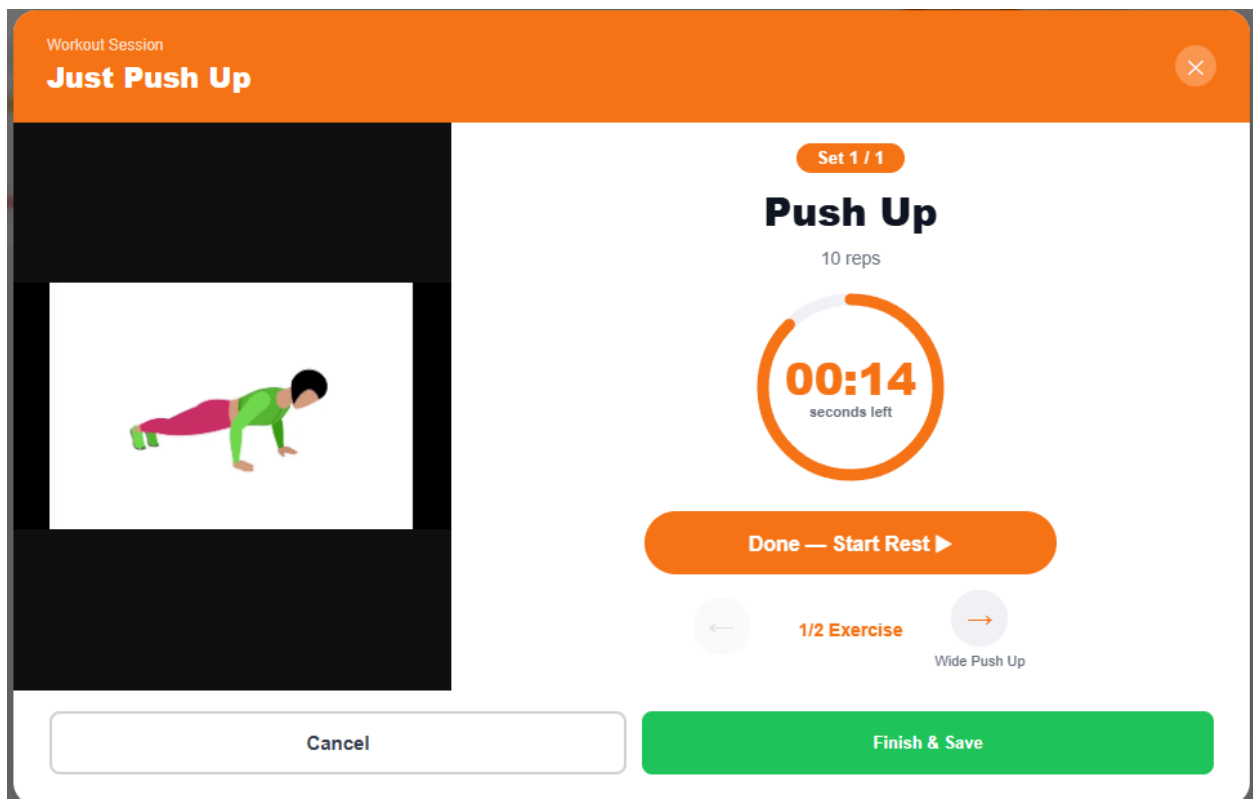
Cancel

Save Workout

Progress Tracker



Workout Session Modal



Workout History Page

BoduBodu
Fitness Tracker

MAIN

- Dashboard
- Exercises
- Workouts
- Custom Workouts

TRACK

- Progress
- Workout History**
- Profile

[+] Logout

Good morning, Rhegynne 🌞
Friday, May 22 · Let's push your limits today.

Workout History

Every session you've completed.

2
TOTAL

1
DEFAULT

1
CUSTOM

9
TOTAL MINS

AllDefaultCustom

MAY 20262 sessions

Casual
May 21, 2026CUSTOM7 min

Leg
May 21, 2026DEFAULT2 min

Profile Page

BoduBodu
Fitness Tracker

MAIN

- Dashboard
- Exercises
- Workouts
- Custom Workouts

TRACK

- Progress
- Workout History
- Profile**

[+] Logout

Good morning, Rhegynne 🌞
Friday, May 22 · Let's push your limits today.

My Profile

Account information

FULL NAME
Rhegynne Hisoler

EMAIL ADDRESS
hisolerrhagynne@gmail.com

MEMBER SINCE
April 10, 2026

PASSWORD

.....

Minimum 8 characters

Change

Update account

Fitness summary

2
total workouts

9
total minutes

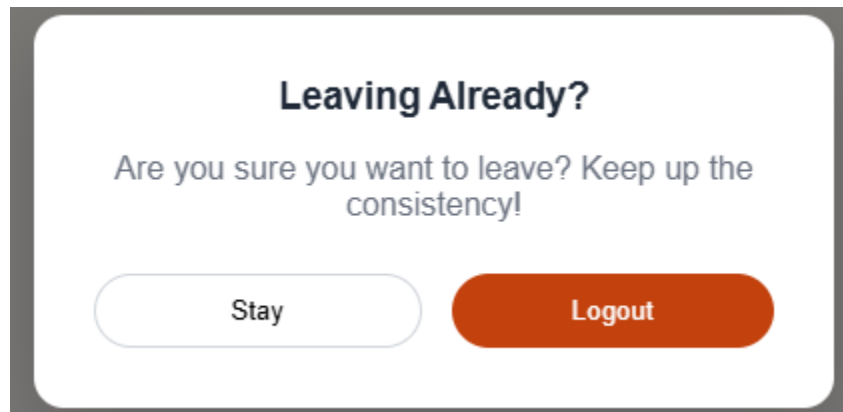
1d
current streak

2
this week

View full history →

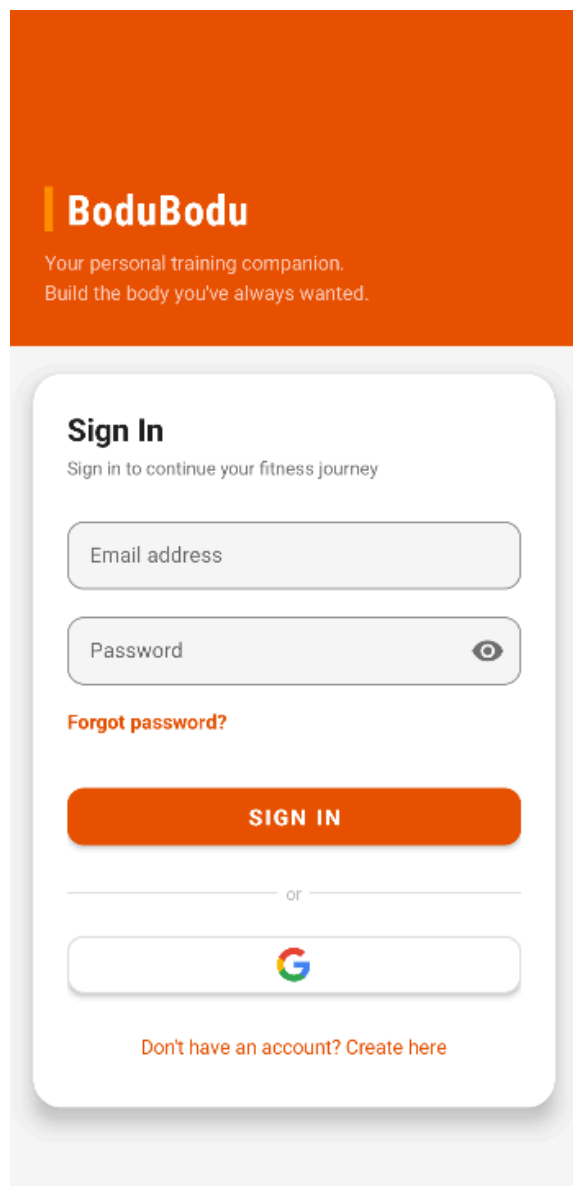
24

Logout Modal



7.2 Mobile Application Wireframes / Screenshots

Login Screen



Register Screen

BoduBodu

Create Account

Join thousands building stronger bodies with bodyweight training.

Personal Details

First name

Last name

Account Details

Email address

Password

Use at least 6 characters

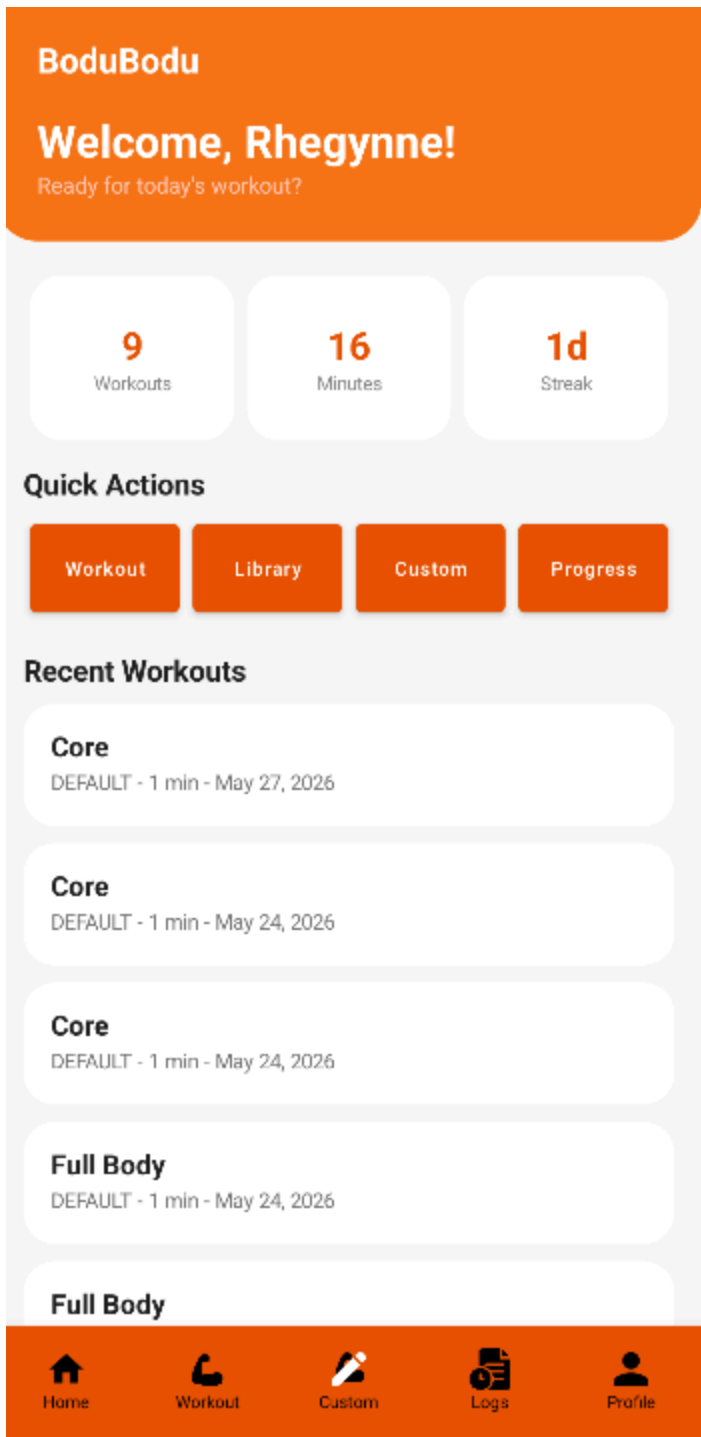
Confirm password

By registering, you agree to our Terms of Service and Privacy Policy.

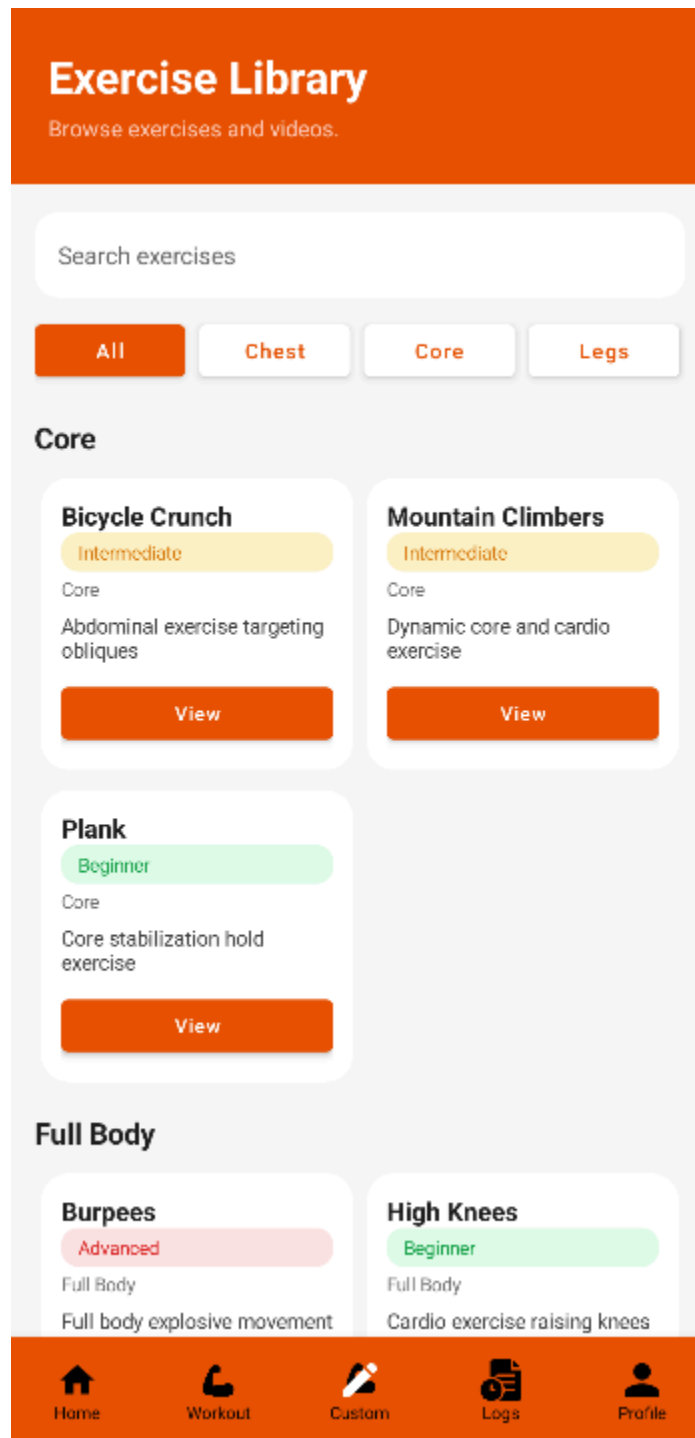
CREATE ACCOUNT

Already have an account? Sign in

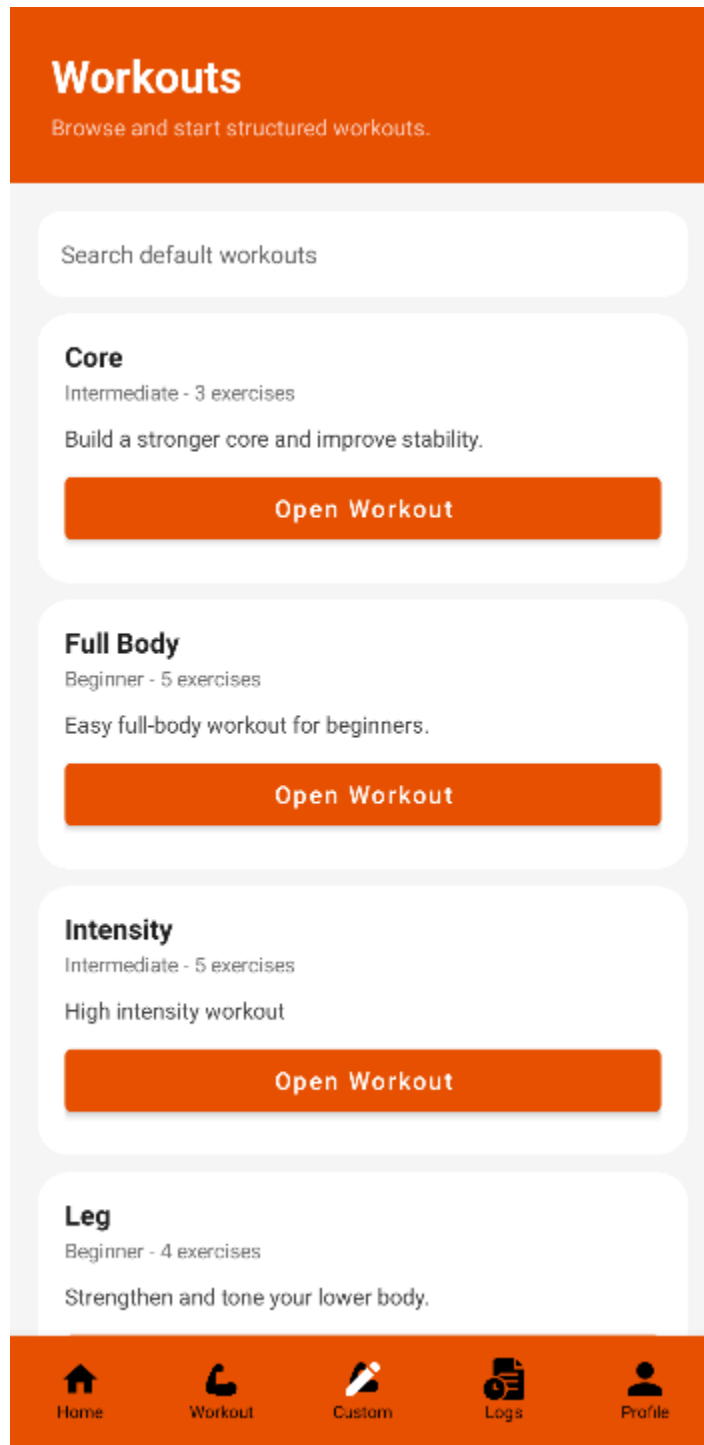
Dashboard/Home Screen



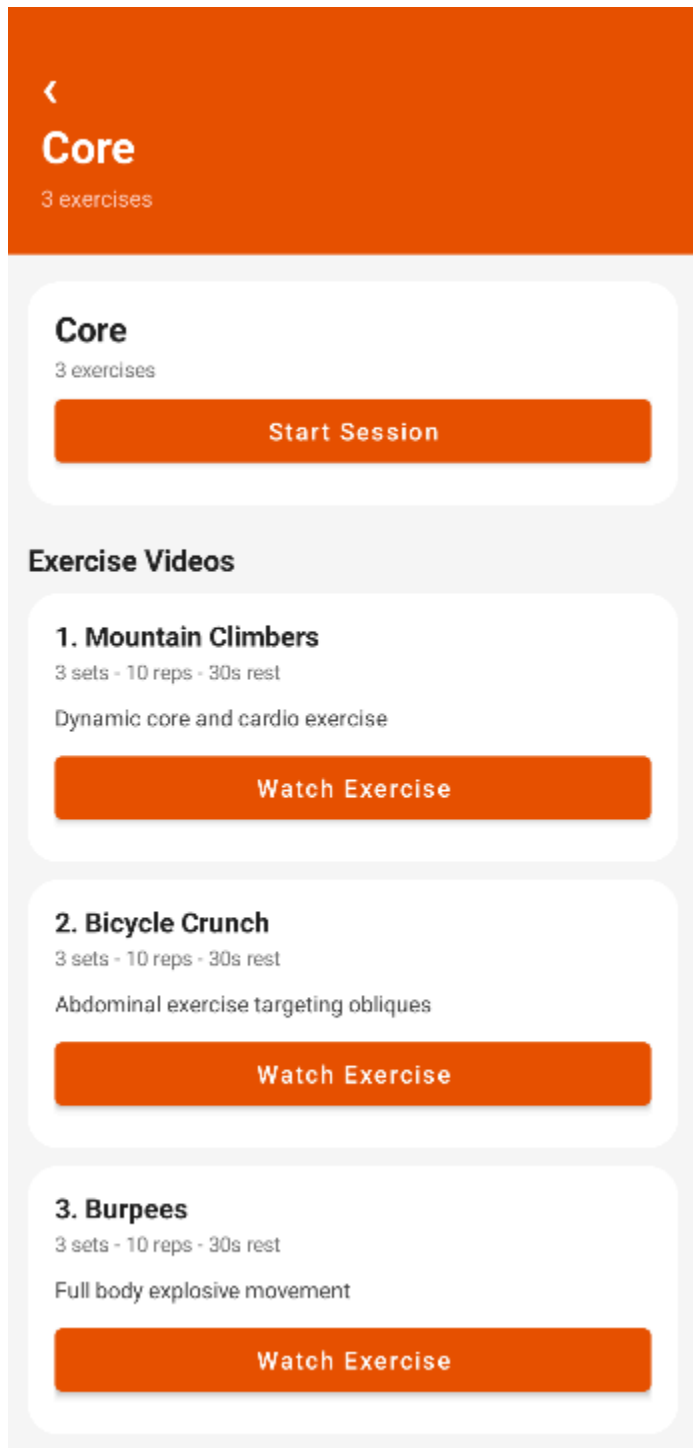
Exercise Library Screen



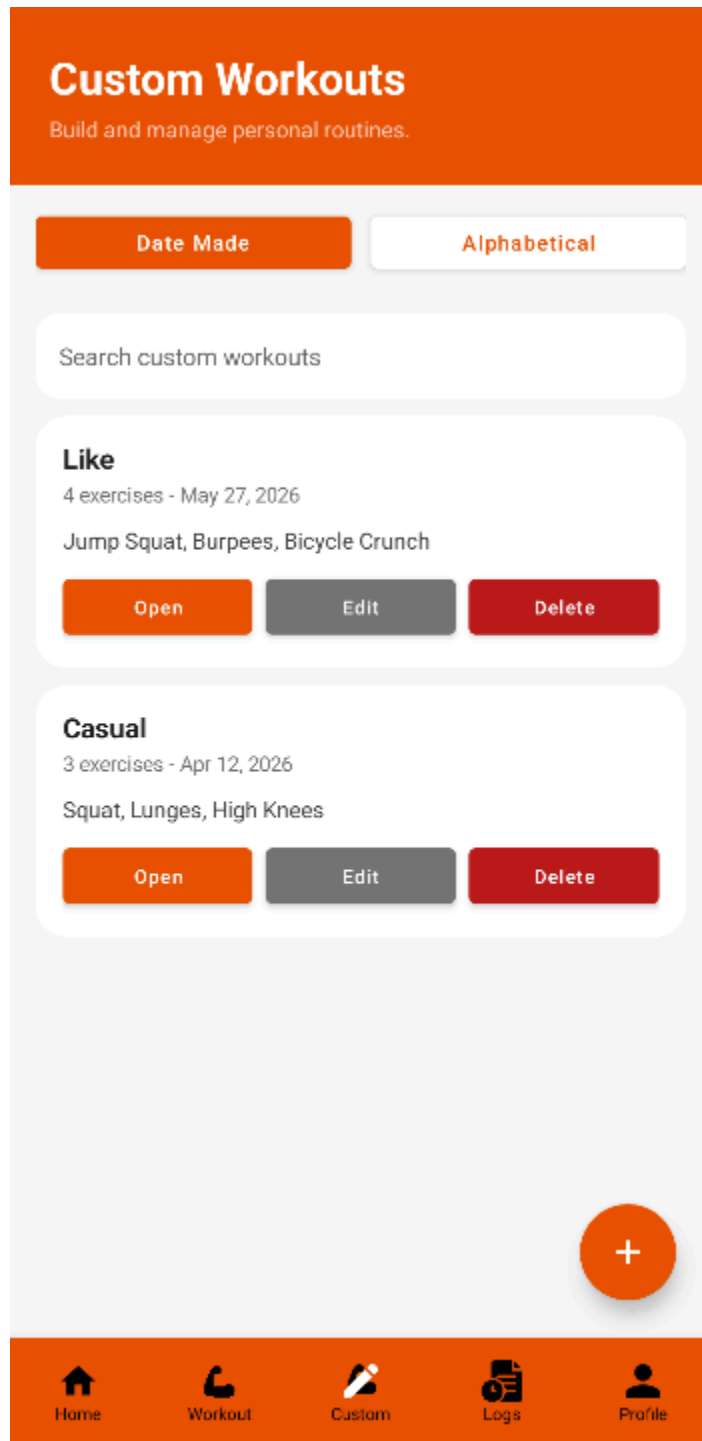
Workout Screen



Workout Details Screen



Custom Workout Screen



Workout Session



Core

Set 1 / 3



Mountain Climbers

10 reps

Start

00:07
seconds left

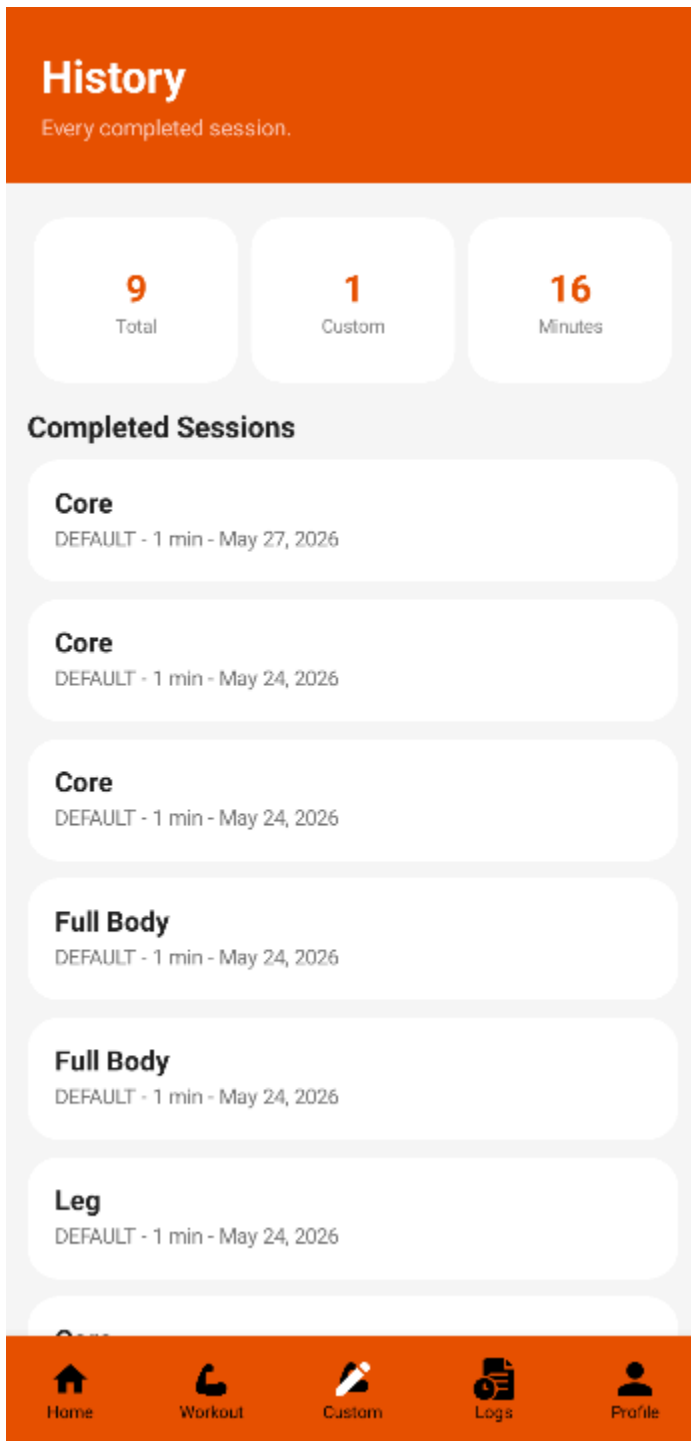
>

Bicycle Crunch

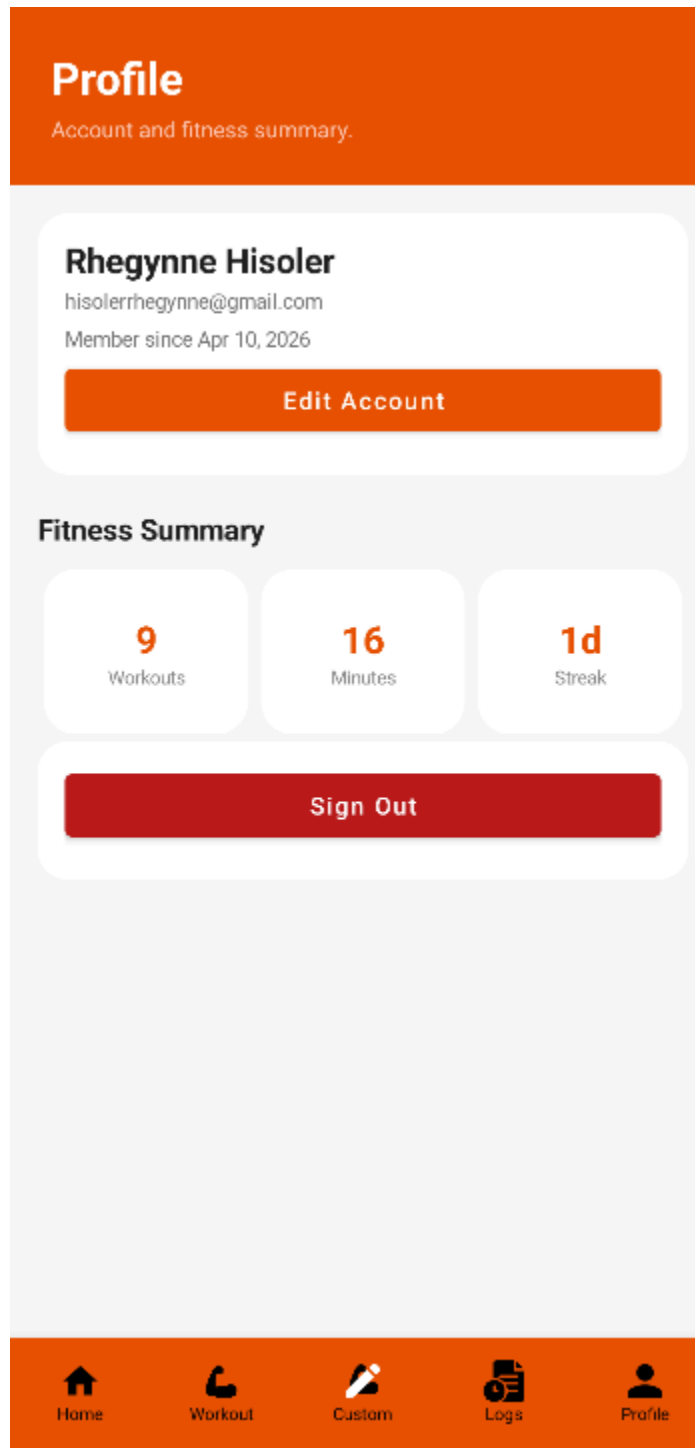
Done - Start Rest

Finish & Save

Workout History Screen



Profile Screen



8.0 PLAN AND CURRENT IMPLEMENTATION STATUS

8.1 Project Timeline and Implementation Status

Phase 1: Planning & Design (Week 1–2)

Week 1: Requirements & Architecture

- Project setup and initial documentation
- Completion of Functional Requirements (FRS) and Non-Functional Requirements (NFR)
- System architecture planning and overall structure design

Week 2: Detailed Design

- API endpoint specification
- PostgreSQL database design and implemented JPA entity review
- UI/UX wireframe preparation
- Implementation plan finalization

Phase 2: Backend Development (Week 3–4)

Week 3: Foundation

- Spring Boot project setup with required dependencies
- Database configuration and entity creation (users, exercise, default/custom workouts, workout sessions)
- Implementation of authentication using PasswordEncoder, JWT, Google auth, and password reset support
- Development of exercise CRUD endpoints

- Development of predefined workout endpoints

Week 4: Core Features

- Implementation of custom workout creation logic
- Development of workout session tracking
- Implementation of user profile, dashboard statistics, recent workouts, and workout history endpoints
- Input validation and error handling improvements
- API testing and refinement using Postman

Phase 3: Web Application Development (Week 5–6)

Week 5: Frontend Foundation

- ReactJS project setup
- Development of authentication pages (register and login)
- Implementation of exercise listing page
- Development of workout listing and detail pages
- Creation of custom workout builder interface

Week 6: Complete Web Features

- Implementation of workout session execution page
- Development of profile and workout history page
- Admin user, exercise, and predefined workout management interface

- Responsive design adjustments
- API integration testing and debugging

Phase 4: Mobile Application Development (Week 7–8)

Week 7: Android Foundation

- Android project setup and structure configuration
- Development of authentication screens
- Implementation of exercise browsing screens
- Workout viewing and execution functionality
- API integration using Retrofit and Gson converter

Week 8: Complete Mobile App

- Custom workout creation feature
- Workout history and profile screen
- UI refinement and improvements
- Testing on emulator and physical device
- APK generation and documentation

Phase 5: Integration & Finalization (Week 9–10)

Week 9: Integration Testing

- End-to-end testing across web and mobile platforms

- Bug fixing and performance improvements
- Security review and validation checks
- Database and endpoint alignment review
- Documentation updates

Week 10: Finalization & Submission

- Backend final review and cleanup
- Web application final adjustments
- Mobile application final adjustments
- Final system testing
- Project submission preparation

Milestones

- M1: All design documents completed
- M2: Backend API fully functional
- M3: Web application integrated with backend
- M4: Mobile application integrated
- M5: Full system tested and finalized

Critical Path

- Authentication, password reset, and Google sign-in implementation
- Exercise and default/custom workout API completion
- Workout logging, dashboard statistics, and history logic

- Frontend/mobile-backend integration
- Cross-platform testing and documentation

Risk Mitigation

- Start with the simplest working version of each feature
- Test integration points early and often
- Keep backup copies of stable versions
- Focus on core functionality before enhancements